

# Bases de datos

UD8. Sentencias de manipulación de datos en SQL

INSERT, UPDATE, DELETE



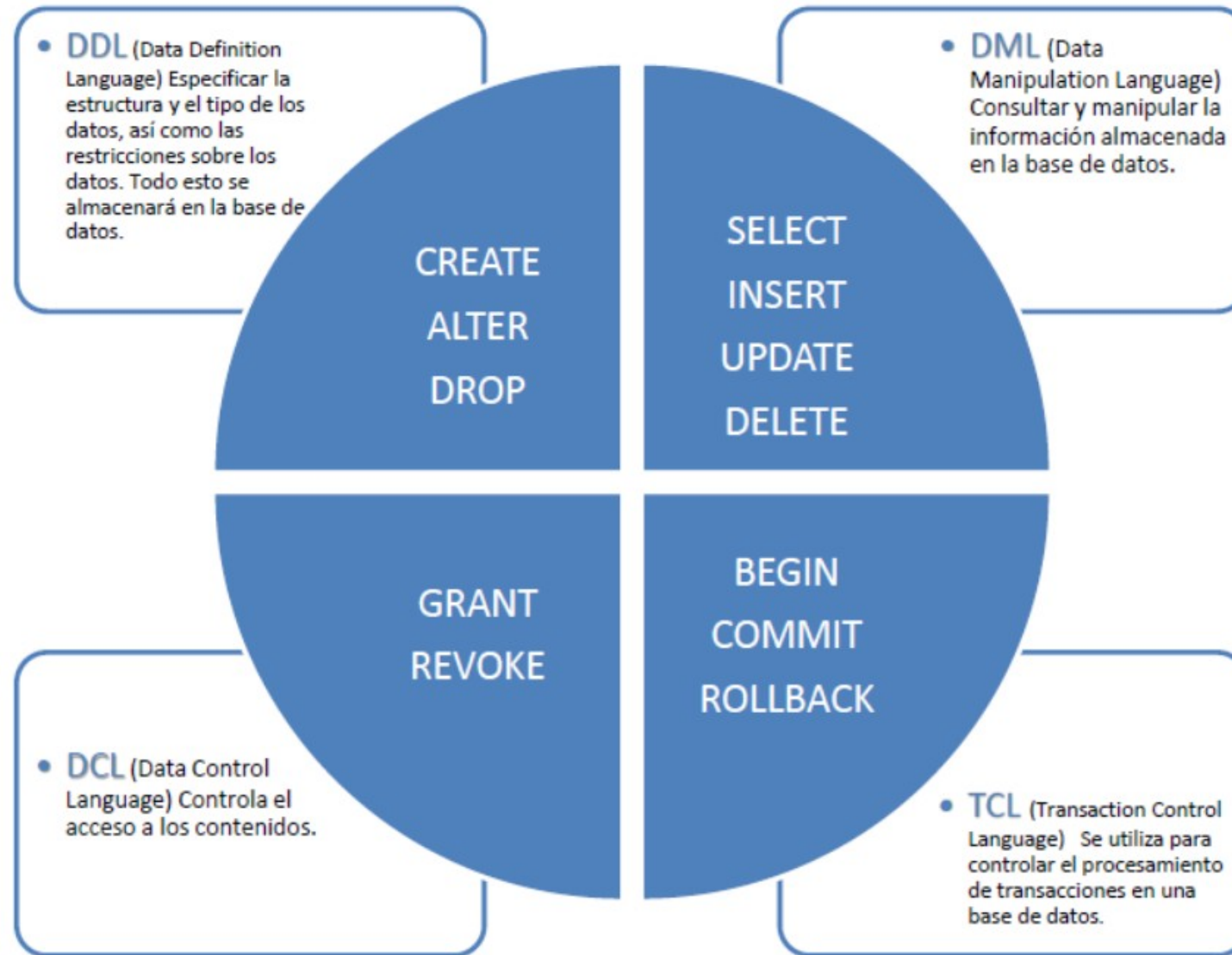
# 1.ÍNDICE

1.INSERT

2.DELETE

3.UPDATE

# COMPONENTES DE LOS SGBD: LENGUAJES



# EL LENGUAJE DML

- ▶ Las sentencias DML del lenguaje SQL son las siguientes:
  - ▶ SELECT: Se utiliza para extraer (proyectar) información de la base de datos, tanto de una tabla, como de varias.
  - ▶ INSERT: Inserta uno o varios registros en una tabla
  - ▶ DELETE: Borra registros de una tabla
  - ▶ UPDATE: Modifica registros en una tabla.
- ▶ A todas las sentencias o comandos que se ejecutan en un SGBD se les denomina CONSULTAS o QUERIES (a veces, también se denominan consultas solo a las sentencias SELECT, por lo que da lugar a errores de interpretación).
- ▶ Todas las sentencias terminan en punto y coma (;).
- ▶ En este tema nos vamos a centrar en las tres últimas.

# TRANSACCIONES

Se considera transacción, a un conjunto de instrucciones o sentencias SQL que se tratan como una sola instrucción, y puede ser confirmada (commit) o abortada (rollback). Se ejecutan todas de forma atómica. Esto significa que, o se hacen todas las operaciones, o no se hace ninguna. Pero hay que tener en cuenta que las sentencias DDL no son parte de las transacciones, y siempre se ejecutan sin necesidad de confirmarse.

Las transacciones son útiles para poder realizar varias operaciones y mantener la integridad de los datos, de forma que o se ejecutan todas, o no se ejecutan ninguna. Por ejemplo, si se quiere enviar un pedido, habrá que anotar el pedido, rellenar la factura, y eliminar los productos del stock. Si primero se anota el pedido, luego se rellena la factura, y a la hora de eliminar los productos del stock, resulta que no hay stock suficiente, con una transacción se puede abortar la transacción, para no anotar el pedido ni haber generado la factura.

# TRANSACCIONES: COMMIT y ROLLBACK

Las sentencias DML que manipulan y alteran datos (INSERT, DELETE y UPDATE) deben persistidas/confirmadas en la base de datos para que otro usuario pueda ver sus efectos en otra sesión. Si no son persistidas/confirmadas :

- ▶ solo en la sesión del usuario que las ha realizado las verá. Ningún otro usuario podrá visualizar los cambios en su sesión.
- ▶ permanecerán en la sesión del usuario que las realizó hasta que salga de la sesión. En ese momento, los cambios no confirmados se desharán.

La instrucción que nos permite hacer permanentes los cambios en una base de datos es **COMMIT**. La mayoría de los SGBD y herramientas tienen por defecto activo el **AUTOCOMMIT**. Cuando esta variable está activa, lo que ocurre es que cada vez que se ejecuta instrucción, se ejecuta un COMMIT de transparentemente al usuario. Esto puede ser beneficioso en ocasiones, aunque es decisión de los analistas y del administrador de la base de datos desactivar el AUTOCOMMIT por defecto si las necesidades de uso de la base de datos lo requieren.

La variable AUTOCOMMIT puede tomar los valores 1/ON cuando está activa 0/OFF cuando está desactivada;

Para ver el valor de la variable utilizamos SHOW:

```
SHOW SESSION VARIABLES like'%commit%';
```

Para desactivarla, utilizamos el comando SET:

```
SET AUTOCOMMIT = 0;  
SET AUTOCOMMIT = OFF;
```

# TRANSACCIONES: COMMIT y ROLLBACK

Si estamos trabajando en una sesión de MySQL con el AUTOCOMMIT desactivado, las transacciones que se realicen se quedan 'pendientes de confirmación' y no se ejecutan en la base de datos hasta que no se haga COMMIT. Si mientras se realizan estos cambios, se necesita retroceder y deshacer las transacciones realizadas desde que se ejecutó el último COMMIT, se puede ejecutar la sentencia ROLLBACK, que deshace todas las operaciones desde la última confirmación.

De esta forma puedo realizar varias operaciones de borrado/inserción/actualización y cuando considere que todo está bien, ejecutar el COMMIT para hacer los cambios definitivos. Si durante este proceso se produce algún error, tengo opción de deshacer los cambios desde el último COMMIT ejecutando la sentencia ROLLBACK.

# TRANSACCIONES: COMMIT y ROLLBACK (Ejemplo)

SET AUTOCOMMIT = 0;

DELETE FROM LIBROS;

INSERT INTO personal (código) VALUES (1);

INSERT INTO personal (código) VALUES (1); → Esta sentencia falla, pero el DELETE anterior es parte de la transacción, y siguen los libros borrados para esta sesión (no para las demás).

DROP TABLE personal; → Esta instrucción lleva implícito el COMMIT, por lo que el ROLLBACK de después no la deshace (ni deshace las anteriores).

INSERT INTO libros (num\_paginas, editorial) SELECT 10, nombre\_e FROM editoriales;

ROLLBACK; → Deshace solo el insert anterior.

DELETE FROM libros WHERE num\_paginas > 50;

UPDATE libros SET num\_paginas = Num\_paginas + 50 WHERE codigo > 100000;

COMMIT; → Aplica el delete y el update para que el resto de sesiones tengan los cambios.



# TRANSACCIONES: ACCESO SIMULTÁNEO A DATOS

Cuando se realizan transacciones, puede haber problemas en el acceso a los datos, porque dos transacciones (en dos sesiones distintas) accedan y manipulen el mismo dato (registro, campo o tabla). Estos problemas son:

- ▶ Lectura sucia (dirty read): Una transacción lee datos que en otra sesión ya han sido modificados, pero como no se ha realizado el commit, no ve estos cambios.
- ▶ Lectura no repetible (Nonrepeatable read): Una transacción vuelve a leer datos que había leído, y se encuentra que han sido modificados por otra transacción.
- ▶ Lectura fantasma (phantom read): Una transacción lee unos datos que no existían cuando se inició la transacción.

Para solucionar estos problemas, el SGBD puede bloquear un conjunto de datos a diferentes niveles (registro, campo, tabla ...). Así, el Gestor bloquea o aísla los datos, y no permite a otras transacciones operar con ellos.

# TRANSACCIONES: ACCESO SIMULTÁNEO A DATOS

Existen cuatro niveles de aislamiento, que definen como los cambios hechos por una transacción son visibles a otras transacciones (de otras sesiones).

- ▶ Lectura no persistida (**Read Uncommitted**): no puede evitar que sucedan los los tres problemas vistos anteriormente. Las sentencias SELECT son efectuadas sin realizar bloqueos, por tanto, todos los cambios hechos por una transacción pueden verlos las otras transacciones.
- ▶ Lectura persistida (**Read committed**): Los datos leídos por una transacción pueden ser modificados por otras transacciones. Se pueden dar los problemas de Phantom Read y Non Repeatable Read.
- ▶ Lectura repetible (**Repeatable Read**); Ningún registro leído con un SELECT se puede cambiar en otra transacción. Tan solo se permite el problema del Phantom Read.
- ▶ **Serializable**: Las transacciones ocurren de forma totalmente aislada a otras transacciones. Se bloquean las transacciones de tal manera que ocurren una detrás de otras, sin capacidad de concurrencia. El SGBD las ejecuta concurrentemente, y previene los interbloqueos.

El nivel de aislamiento en MySQL (en el motor InnoDB) por defecto es Repeatable Read.

# TRANSACCIONES: ACCESO SIMULTÁNEO A DATOS

Existen cuatro niveles de aislamiento, que definen como los cambios hechos por una transacción son visibles a otras transacciones (de otras sesiones).

- ▶ Lectura no persistida (**Read Uncommitted**): no puede evitar que sucedan los los tres problemas vistos anteriormente. Las sentencias SELECT son efectuadas sin realizar bloqueos, por tanto, todos los cambios hechos por una transacción pueden verlos las otras transacciones.
- ▶ Lectura persistida (**Read committed**): Los datos leídos por una transacción pueden ser modificados por otras transacciones. Se pueden dar los problemas de Phantom Read y Non Repetable Read.
- ▶ Lectura repetible (**Repeatable Read**); Ningún registro leído con un SELECT se puede cambiar en otra transacción. Tan solo se permite el problema del Phantom Read.
- ▶ **Serializable**: Las transacciones ocurren de forma totalmente aislada a otras transacciones. Se bloquean las transacciones de tal manera que ocurren una detrás de otras, sin capacidad de concurrencia. El SGBD las ejecuta concurrentemente, y previene los interbloqueos.

El nivel de aislamiento en MySQL (en el motor InnoDB) por defecto es Repeatable Read y el bloqueo por defecto es a nivel de fila.

# INSERT

La sentencia SQL que permite introducir registros en una tabla de la base de datos es **INSERT**.

La sintaxis de la instrucción es:

```
INSERT INTO {nombre_tabla} [(columna1 [, columna2]...)]  
VALUES (valor1 [, valor2] ... );
```

No es necesario introducir todos los valores de los campos en cada registro, lo que sí debe ocurrir es que el número de valores debe corresponderse con el número de columnas definidas, y en el mismo orden, de tal forma que el valor2 se introducirá en la columna2.

También podemos poner en la sección values todos los valores de los campos en orden (insertando a NULL aquellos que no queramos usar), y, en ese caso, podemos obviar la parte de la instrucción donde se detallan las columnas de la tabla.

Veamos un ejemplo en nuestro modelo de libros y editoriales.

# INSERT

- ▶ Imaginemos que la tabla libros está vacía y queremos insertar el primer libro.

```
INSERT  
INTO LIBROS (codigo, titulo, num_paginas, editorial)  
VALUES (34587, 'Int. Artificial', 50, 'Paraninfo');
```



| LIBROS |                 |             |           |
|--------|-----------------|-------------|-----------|
| Codigo | Titulo          | Num_paginas | Editorial |
| 34587  | Int. Artificial | 50          | Paraninfo |

- ▶ Como ya hemos comentado, si vamos a introducir todos los datos del registro, podemos obviar la lista de columnas, pero debemos asegurarnos que el orden y los tipos coinciden. Es una opción que hay que usar con mucho cuidado, pues las columnas de una tabla se pueden alterar en orden y cantidad:

```
INSERT  
INTO LIBROS  
VALUES (1022305, 'Concep. Y Dis', 48, 'Rama');
```



| LIBROS  |                 |             |           |
|---------|-----------------|-------------|-----------|
| Codigo  | Titulo          | Num_paginas | Editorial |
| 34587   | Int. Artificial | 50          | Paraninfo |
| 1022305 | Concep. Y Dis.  | 48          | Rama      |

- ▶ Podemos poner las columnas desordenadas, siempre que luego coincidan con los valores:

```
INSERT  
INTO LIBROS (codigo, titulo, editorial, num_paginas)  
VALUES (493942, 'Turbo C++', 'Mc Graw-Hill', 125);
```



| LIBROS  |                 |             |              |
|---------|-----------------|-------------|--------------|
| Codigo  | Titulo          | Num_paginas | Editorial    |
| 34587   | Int. Artificial | 50          | Paraninfo    |
| 1022305 | Concep. Y Dis.  | 48          | Rama         |
| 493942  | Turbo C++       | 125         | Mc Graw-Hill |

# INSERT

- ▶ Si en algún campo no se indica el valor, se grabará el valor definido con DEFAULT cuando se creó la tabla. Si no se definió ninguno, se insertará NULL. También se puede indicar el campo y darle el valor NULL. **Ojo, si el campo se definió como NOT NULL en la creación de la tabla, dará error.**

Estas tres  
sentencias  
hacen la  
misma  
inserción  
en esta  
tabla.

```
INSERT  
INTO LIBROS (codigo, titulo, num_paginas)  
VALUES (45307, 'Virus Informát.', 50);
```

```
INSERT  
INTO LIBROS (codigo, titulo, num_paginas, Editorial)  
VALUES (45307, 'Virus Informát.', 50, NULL);
```

```
INSERT  
INTO LIBROS VALUES (45307, 'Virus Informát.', 50, NULL);
```



| LIBROS  |                 |             |              |
|---------|-----------------|-------------|--------------|
| Codigo  | Titulo          | Num_paginas | Editorial    |
| 34587   | Int. Artificial | 50          | Paraninfo    |
| 1022305 | Concep. Y Dis.  | 48          | Rama         |
| 493942  | Turbo C++       | 125         | Mc Graw-Hill |
| 45307   | Virus Informát. | 50          | NULL         |

- ▶ Podemos insertar cualquier expresión, siempre y cuando el resultado de la misma sea del tipo de datos de la columna en la que lo estamos insertando.

```
INSERT  
INTO LIBROS (codigo, titulo, num_paginas, editorial)  
VALUES (112000+313, CONCAT('Int. ' , 'Artificial'), 100-50, 'Paraninfo');
```



| LIBROS  |                 |             |              |
|---------|-----------------|-------------|--------------|
| Codigo  | Titulo          | Num_paginas | Editorial    |
| 34587   | Int. Artificial | 50          | Paraninfo    |
| 1022305 | Concep. Y Dis.  | 48          | Rama         |
| 493942  | Turbo C++       | 125         | Mc Graw-Hill |
| 45307   | Virus Informát. | 50          | NULL         |
| 112313  | Sist. Informac. | 358         | Rama         |

# INSERT: Valores DEFAULT

Como hemos visto, si a una columna no se le indica el valor que va a tener a la hora de insertar un registro, pueden ocurrir dos cosas:

- ▶ Que se le asigne el valor que tiene definido por defecto, Bien por no indicarlo o forzándolo con la palabra DEFAULT.
- ▶ Que se quede con el campo vacío (NULL)

```
CREATE TABLE LIBROS (  
Codigo INT PRIMARY KEY AUTO_INCREMENT,  
Titulo VARCHAR(20) NOT NULL,  
Num_paginas INT DEFAULT 0,  
Editorial VARCHAR(30)  
);
```

Imaginemos que acabamos de crear la tabla definida en el recuadro y está vacía. Veamos algunos ejemplos de uso de DEFAULT en esta tabla:

| Sentencia                                                                                        | Explicación                                                                                                            | Resultado                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INSERT INTO<br>LIBROS (Titulo, Editorial)<br>VALUES ('Int. Artificial', 'Paraninfo');            | En este caso, el id se rellena porque es autoincremental y el número de páginas se rellenará con el valor por defecto. | <pre>+-----+-----+-----+-----+<br/>  Codigo   Titulo             Num_paginas   Editorial  <br/>+-----+-----+-----+-----+<br/>       1   Int. Artificial              0   Paraninfo  <br/>+-----+-----+-----+-----+</pre> |
| INSERT INTO<br>LIBROS (Codigo, Titulo, Num_paginas)<br>VALUES (125, 'Virus Informát.', DEFAULT); | En este caso, se introduce el código, se pone por defecto a 0 el número de páginas, y la Editorial se queda a NULL     | <pre>+-----+-----+-----+-----+<br/>  Codigo   Titulo             Num_paginas   Editorial  <br/>+-----+-----+-----+-----+<br/>     125   Virus Informát.             0   NULL       <br/>+-----+-----+-----+-----+</pre>  |

# INSERT : Columnas AUTO\_INCREMENT

En MySQL, las columnas AUTO\_INCREMENT son un tipo especial de columna numérica de enteros (INT, BIGINT, SMALLINT, TINYINT) utilizada para generar automáticamente valores únicos para cada fila nueva en una tabla. Estas columnas suelen ser clave primaria o formar parte de una clave única. Se define en el momento de creación de la tabla.

```
CREATE TABLE tabla (  
id INT AUTO_INCREMENT PRIMARY KEY,  
campo1 VARCHAR(50));
```

Cada vez que se inserta una nueva fila en la tabla, MySQL incrementa automáticamente el valor de la columna en 1 (o el valor definido como incremento) sin que sea necesario gestionarlo explícitamente de ninguna forma en la consulta de inserción.

Por ejemplo, en la tabla anterior, si queremos insertar un registro, tenemos varias opciones:

| Opción | Sentencia                                                                     | Explicación                                                                                                                     | Resultado                                                                                                                                  |
|--------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| op1    | <b>INSERT INTO tabla (campo1)<br/>VALUES ('Valor del registro 1');</b>        | No definimos el campo id, por lo que lo iniciará en 1 directamente.                                                             | <pre>select * from tabla;<br/>+-----+-----+<br/>  id   campo1  <br/>+-----+-----+<br/>  1   Valor del registro 1  <br/>+-----+-----+</pre> |
| op2    | <b>INSERT INTO tabla (id, campo1)<br/>VALUES (1, 'Valor del registro 1');</b> | Definimos el valor del campo id. El valor de la secuencia se queda preparado para el siguiente valor a insertar, que será el 2. |                                                                                                                                            |



# INSERT : Columnas AUTO\_INCREMENT

Si ahora insertamos el siguiente registro en la misma tabla, tendremos que hacerlo respectivamente:

| Opción | Sentencia                                                                     | Explicación                                                                                                                                                                                                                                                                        | Resultado                                                                                                                                  |
|--------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| op1    | <b>INSERT INTO tabla (campo1)<br/>VALUES ('Valor del registro 2');</b>        | No definimos el campo id, por lo que directamente se rellenará con el siguiente valor de la secuencia, el 2.                                                                                                                                                                       | <pre>select * from tabla; +----+-----+   id   campo1   +----+-----+   1   Valor del campo 1     2   Valor del campo 2   +----+-----+</pre> |
| op2    | <b>INSERT INTO tabla (id, campo1)<br/>VALUES (2, 'Valor del registro 2');</b> | Definimos el valor del campo id. El valor de la secuencia ha de ser diferente de 1 y, después de la inserción, se queda preparado para el siguiente valor a insertar, que será el 3. Si hubiésemos insertado el 7, la secuencia se quedaría preparada en el siguiente valor, el 8. |                                                                                                                                            |

Si borramos registros de las tablas, las secuencias se ven alteradas, es decir no se reinician. Por ejemplo, si ahora borramos el registro con id = 2, el siguiente valor que insertará será el 3, no se decrementa al borrar.

Si realizamos un TRUNCATE de la tabla, el valor del campo AUTO\_INCREMENT de una tabla SI se reinicia a 1. Pero si borramos el contenido de una tabla usando DELETE, el campo AUTO\_INCREMENT NO se reinicia, aunque hayamos borrado todos los registros de la tabla.

# INSERT : Columnas AUTO\_INCREMENT

La ventaja es que garantizan valores únicos para la clave primaria. Pero en su manejo hay que tener en cuenta que:

- ▶ Solo puede haber una columna AUTO\_INCREMENT por tabla.
- ▶ Si eliminas la última fila, el valor AUTO\_INCREMENT no se reduce automáticamente. Se puede ajustar manualmente el siguiente número de la secuencia usando la sentencia.

```
ALTER TABLE tabla AUTO_INCREMENT = valor_deseado;
```

Por defecto, todas las columnas definidas como auto\_increment comienzan en el 1 y se incrementan de 1 en uno. Podemos cambiar esto usando alguna de las siguientes sentencias:

```
SET @@auto_increment_increment = 2;    -- Incremento de 2 en 2
SET @@auto_increment_offset = 5;       -- El primer valor de la secuencia es 5*
```

\*Si se ejecuta `SET @@auto_increment_offset=5` una vez la tabla está creada, no se efectúa el cambio. Si se crea la tabla después, si. Para evitar problemas, si la tabla ya está creada, usará la sentencia `ALTER TABLE tabla AUTO_INCREMENT = 5`, que tendrá el mismo efecto.

```
CREATE TABLE LIBROS (  
Codigo INT PRIMARY KEY AUTO_INCREMENT,  
Titulo VARCHAR(20) NOT NULL,  
Num_paginas INT DEFAULT 0,  
Editorial VARCHAR(30) );
```

# INSERT Extendido o múltiple

Algunos SGBD como MySQL permiten una sintaxis extendida para insertar más de una fila a la vez. Para ello, la lista de valores aparece más de una vez (cada vez con valores de una fila) separándose por comas.

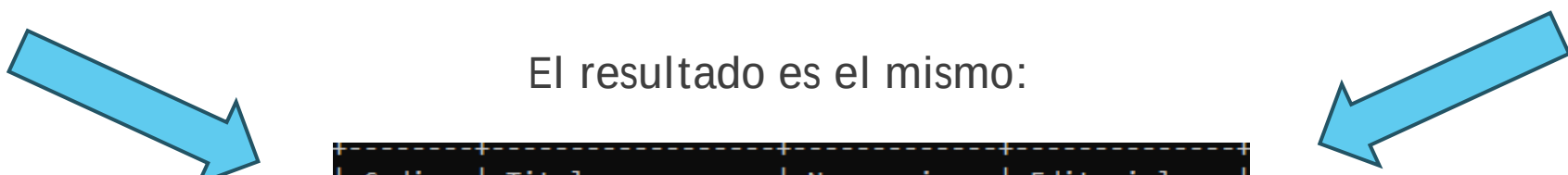
Podemos insertar los libros con sentencias INSERT:

```
INSERT INTO LIBROS (titulo, num_paginas, editorial)  
VALUES ('Int. Artificial', 50, 'Paraninfo');  
INSERT INTO LIBROS (titulo, num_paginas, editorial)  
VALUES ('Virus Informát. ', 50, NULL);  
INSERT INTO LIBROS (titulo, num_paginas, editorial)  
VALUES ('Sist. Informac. ', NULL, 'Rama');  
INSERT INTO LIBROS (titulo, num_paginas, editorial)  
VALUES ('Turbo C++', NULL, 'Mc Graw-Hill');
```

O podemos insertar los cuatro libros cuatro usando la misma sentencia:

```
INSERT INTO  
LIBROS (titulo, num_paginas, editorial)  
VALUES ('Int. Artificial', 50, 'Paraninfo'),  
('Virus Informát. ', 50, NULL),  
('Sist. Informac. ', NULL, 'Rama'),  
('Turbo C++', NULL, 'Mc Graw-Hill');
```

El resultado es el mismo:



| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

```
CREATE TABLE LIBROS (  
Codigo INT PRIMARY KEY AUTO_INCREMENT,  
Titulo VARCHAR(20) NOT NULL,  
Num_paginas INT DEFAULT 0,  
Editorial VARCHAR(30) );
```

# INSERT Extendido o múltiple

Pero hay que ser cuidadoso cuando usamos el INSERT extendido, porque si alguna de las filas falla, no se inserta ninguna de las otras, aunque las otras estén bien. Es una instrucción que se realiza de forma atómica, o se insertan todas o ninguna. Sin embargo, de la otra forma fallará tan sólo un INSERT, y los otros tres se realizarán correctamente.

```
INSERT INTO  
LIBROS (titulo, num_paginas, editorial)  
VALUES ('Int. Artificial', 50, 'Paraninfo'),  
      ('Virus Informát. ', 50, NULL),  
      ('Sist. Informac. ', NULL, 'Rama', 'Fila con una columna mas que dará error'),  
      ('Turbo C++', NULL, 'Mc Graw-Hill');
```



```
mysql> INSERT INTO  
-> LIBROS (titulo, num_paginas, editorial)  
-> VALUES ('Int. Artificial', 50, 'Paraninfo'),  
->          ('Virus Informát. ', 50, NULL),  
->          ('Sist. Informac. ', NULL, 'Rama', 'Fila con una columna mas que dará error'),  
->          ('Turbo C++', NULL, 'Mc Graw-Hill');  
ERROR 1136 (21S01): Column count doesn't match value count at row 3  
mysql> select * from LIBROS;  
Empty set (0.00 sec)
```

```
CREATE TABLE LIBROS (  
Codigo INT PRIMARY KEY AUTO_INCREMENT,  
Titulo VARCHAR(20) NOT NULL,  
Num_paginas INT DEFAULT 0,  
Editorial VARCHAR(30) );
```

# INSERT Restricciones y errores

Hemos visto que el INSERT puede dar errores aunque esté correctamente escrito. Puede que alguna columna no se corresponda con el tipo correspondiente pero también ha de cumplir otras restricciones del modelo:

- ▶ Restricción NOT NULL. Si intentamos insertar un valor NULL en un campo que está declarado NOT NULL, dará error.
- ▶ Restricción PRIMARY KEY. Si intentamos insertar un valor NULL en el campo que es PK, dará error.
- ▶ Restricción CHECK. Si intentamos insertar un valor que no cumple una condición CHECK para esa columna, dará error.
- ▶ Restricción UNIQUE. Si intentamos insertar un valor que no cumple una condición UNIQUE para esa columna, dará error.
- ▶ Restricción FOREIGN KEY. Si intentamos insertar un valor en una FK que no se encuentra en la lista de valores de la PK de la tabla a la que hace referencia, dará error.

```
CREATE TABLE LIBROS (  
Codigo INT PRIMARY KEY AUTO_INCREMENT,  
Titulo VARCHAR(20) NOT NULL,  
Num_paginas INT DEFAULT 0,  
Editorial VARCHAR(30) );
```

# INSERT desde SELECT

Podemos insertar en una tabla los registros del resultado de la ejecución de una consulta. De esta forma se insertarán TODAS las filas resultado de la consulta.

Al igual que cuando introducimos valores manualmente, debe existir concordancia de tipos, igual que en cualquier INSERT. Es decir, cada columna resultado de la SELECT debe coincidir en tipos y lugar con las columnas indicadas en el INSERT. Se deben cumplir también todas las restricciones.

La sentencia SELECT puede ser tan compleja como se desee.

```
INSERT INTO {nombre_tabla} [(columna1 [, columna2]...)]  
(SELECT .....);
```

# INSERT desde SELECT

Vamos a ver un ejemplo. Para ello acabamos de crear nuestro modelo.

Creamos la tabla EDITORIALES y la rellenamos:

```
CREATE TABLE EDITORIALES (  
  Nombre_e VARCHAR(30) PRIMARY KEY,  
  Direccion VARCHAR(50),  
  Pais VARCHAR(30),  
  Ciudad VARCHAR(50));
```

Y después la FOREIGN KEY en LIBROS :

```
ALTER TABLE LIBROS  
ADD CONSTRAINT ed_fk FOREIGN KEY (Editorial)  
REFERENCES EDITORIALES(Nombre_e)  
ON DELETE SET NULL ON UPDATE CASCADE;
```

Ahora, rellenamos la tabla EDITORIALES y después la de LIBROS.

```
INSERT INTO EDITORIALES VALUES ('Universal Books','Brown Sq. 23','EEUU', 'Los Ángeles');  
INSERT INTO EDITORIALES VALUES ('Rama', 'Canillas, 144', 'España', 'Madrid');  
INSERT INTO EDITORIALES VALUES ('Mc Graw-Hill', 'Basauri, 17', 'España', 'Madrid');  
INSERT INTO EDITORIALES VALUES ('Paraninfo', 'Virtudes, 7', 'España', 'Madrid');
```

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

```
INSERT INTO  
LIBROS (titulo, num_paginas, editorial)  
VALUES ('Int. Artificial', 50, 'Paraninfo'),  
('Virus Informát.', 50, NULL),  
('Sist. Informac.', NULL, 'Rama'),  
('Turbo C++', NULL, 'Mc Graw-Hill');
```

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

# INSERT desde SELECT

Vamos a crear una tabla de Países y la vamos a rellenar con los diferentes países de la tabla EDITORIALES:

```
CREATE TABLE PAISES (  
  Pais VARCHAR(30) PRIMARY KEY);
```

```
INSERT INTO PAISES (Pais)  
(SELECT DISTINCT Pais FROM EDITORIALES);
```

```
select * from PAISES;  
+-----+  
| Pais  |  
+-----+  
| EEUU  |  
| España |  
+-----+
```



# DELETE

La sentencia SQL que permite borrar uno o más registros completos en una tabla de la base de datos es **DELETE**.

La sintaxis de la instrucción es:

```
DELETE FROM nombre_tabla  
[ WHERE filtro ];
```

**OJO: Si no se utiliza el filtro (la sección WHERE) , todos los registros se modificarán, lo que puede causar cambios masivos no deseados. Siempre es conveniente una copia de seguridad antes de actualizaciones masivas.**

Ejemplos:

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

Borra todos los registros

```
DELETE FROM LIBROS;
```

```
select * from LIBROS;  
Empty set (0.00 sec)
```

```
DELETE FROM LIBROS  
WHERE Num_paginas IS NULL;
```

Borra todos los registros para los que el número de paginas es vacío (NULL)

| Codigo | Titulo          | Num_paginas | Editorial |
|--------|-----------------|-------------|-----------|
| 1      | Int. Artificial | 50          | Paraninfo |
| 2      | Virus Informát. | 50          | NULL      |

# DELETE: Ejemplo con subconsulta

Utilizamos el modelo que hemos creado de LIBROS y EDITORIALES.

EDITORIALES :

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

LIBROS :

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

Ahora, vamos a borrar de la tabla LIBROS aplicando un filtro se subconsulta sobre la tabla EDITORIALES.

```
DELETE FROM LIBROS L  
WHERE L.Editorial IN (SELECT E.Nombre_e  
                      FROM EDITORIALES E  
                      WHERE E.Pais = 'España');
```

Borra todos los LIBROS de editoriales españolas



| Codigo | Titulo          | Num_paginas | Editorial |
|--------|-----------------|-------------|-----------|
| 2      | Virus Informát. | 50          | NULL      |

# DELETE: Restricciones

Hay que tener en cuenta que pueden existir restricciones de integridad referencial (foreign key) que indicarán el comportamiento y permitirán borrar o no.

- ▶ Si son de tipo NO ACTION (en MySQL no funciona) no permite borrar.
- ▶ Si son de tipo RESTRICT no permite borrar y da un error.
- ▶ Si son de tipo CASCADE, no solo borra en la tabla indicada, sino que intenta borrar en la tabla de la referencia. Si no puede borrar en esa otra tabla, da un error.
- ▶ Si son de tipo SET NULL, aplica este cambio en la tabla referenciada.

El borrado, o borra todas las filas indicadas o no borra ninguna. Es decir, es todo parte de la misma acción.

# TRUNCATE (Truncado de tablas)

El lenguaje SQL contiene una sentencia que permite borrar TODO el contenido de una tabla de forma más rápida y eficiente, pero se han de eliminar todos los registros, no se puede aplicar ningún filtro. Esta sentencia es el TRUNCATE, que realiza el truncado de la tabla.

```
TRUNCATE TABLE nombre_tabla;
```

Lo único que previene a esta sentencia de eliminar registros son las restricciones de integridad (FK), es decir, si hay una FK referenciando a esta tabla no nos permitirá truncarla.

Es una sentencia del Lenguaje de Definición de Datos (DDL) por lo que no se puede hacer ROLLBACK (no se puede deshacer).

El truncado de tablas, además de eliminar los datos, es más eficiente puesto que libera memoria reduciendo el tamaño del fichero físico de la base de datos, los índices,...

# DELETE vs TRUNCATE

| Característica                                | DELETE                                                               | TRUNCATE                                                         |
|-----------------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------|
| Tipo de operación                             | DML (Data Manipulation Language)                                     | DDL (Data Definition Language)                                   |
| Uso de <code>WHERE</code>                     | Sí, permite eliminar registros específicos.                          | No, borra toda la tabla.                                         |
| Velocidad                                     | Más lento, elimina fila por fila.                                    | Más rápido, borra la tabla de una vez.                           |
| Genera registros en logs                      | Sí, genera logs por cada fila eliminada.                             | No genera logs por cada fila eliminada, solo registra la acción. |
| Liberación de espacio                         | No libera automáticamente el espacio, solo lo marca como disponible. | Libera espacio en disco de inmediato.                            |
| Autoincrement                                 | No reinicia el contador de <code>AUTO_INCREMENT</code> .             | Reinicia el contador de <code>AUTO_INCREMENT</code> .            |
| Uso de <code>ROLLBACK</code> en transacciones | Sí, se puede deshacer con <code>ROLLBACK</code> .                    | No, al ser una operación DDL, no se puede deshacer.              |

# UPDATE

La sentencia **UPDATE** permite modificar el contenido de los campos de una tabla. Se puede modificar el contenido de un mismo campo en varios (incluso todos) los registros. También se pueden modificar los valores de diferentes campos en la misma sentencia.

La sintaxis de la instrucción es:

```
UPDATE nombre_tabla,  
[nombre_tabla2, ...]  
SET asignaciones  
[WHERE condición]  
[ORDER BY ...]  
[LIMIT filas];
```

En la sección **asignaciones** se indican los valores que queremos dar a determinados campos. El formato es: campo = valor, tantas veces como campos se necesitan actualizar, separando con comas los distintos campos. Como valor de la expresión, también se puede poner DEFAULT.

**OJO: Si no se utiliza el filtro (la sección WHERE) , todos los registros se modificarán, lo que puede causar cambios masivos no deseados. Siempre es conveniente una copia de seguridad antes de actualizaciones masivas.**

# UPDATE

Ejemplos:

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |



Actualiza todos los registros

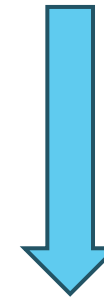
```
UPDATE LIBROS
SET Num_paginas = Num_paginas + 10,
    Editorial = NULL;
```



| Codigo | Titulo          | Num_paginas | Editorial |
|--------|-----------------|-------------|-----------|
| 1      | Int. Artificial | 60          | NULL      |
| 2      | Virus Informát. | 60          | NULL      |
| 3      | Sist. Informac. | NULL        | NULL      |
| 4      | Turbo C++       | NULL        | NULL      |



```
UPDATE LIBROS
SET Num_paginas = Num_paginas + 10,
    Editorial = NULL
WHERE Num_paginas IS NOT NULL;
```



Actualiza sólo los valores en los registros que cumplen la condición del filtro (WHERE)

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 60          | NULL         |
| 2      | Virus Informát. | 60          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

# DELETE Y UPDATE CON MÁS DE UNA TABLA

Se pueden realizar actualizaciones y borrados utilizando los valores de otras tablas.

Pero no es estándar, y en cada SGBD se realiza de forma distinta.

En MySQL, cuando se necesita eliminar registros de una o varias tablas relacionadas, se puede usar DELETE con JOIN.

```
DELETE t1, t2  
FROM tabla1 t1 JOIN tabla2 t2 ON  
t1.pk= t2.fk  
WHERE t2.campo = <valor>;
```

En MySQL, se pueden actualizar registros de una utilizando datos de otra tabla con JOIN.

```
UPDATE tabla1 t1 JOIN tabla2 t2  
ON t1.pk= t2.fk  
SET t1.campo = <valor>;
```



# DELETE Y UPDATE CON MÁS DE UNA TABLA

Veamos un par de ejemplos.

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

Borra en las dos tablas

```
DELETE L, E
FROM LIBROS L JOIN EDITORIALES E
ON E.Nombre_e = L.Editorial
WHERE E.Nombre_e = 'Rama';
```

| Nombre_e        | Direccion    | Pais   | Ciudad      |
|-----------------|--------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17  | España | Madrid      |
| Paraninfo       | Virtudes, 7  | España | Madrid      |
| Universal Books | Brown Sq. 23 | EEUU   | Los Ángeles |

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

```
DELETE L
FROM LIBROS L JOIN EDITORIALES E
ON E.Nombre_e = L.Editorial
WHERE E.Nombre_e = 'Rama';
```

Borra solo en la tabla LIBROS

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

# DELETE Y UPDATE CON MÁS DE UNA TABLA

Veamos un par de ejemplos.

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |



```
UPDATE LIBROS L
JOIN EDITORIALES E
ON E.Nombre_e = L.Editorial
SET L.Num_paginas = 1000
WHERE E.Nombre_e = 'Rama';
```



| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | 1000        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

Actualiza con el valor 1000 el número de páginas en la tabla LIBROS para aquellos que son de la editorial Rama.

# DELETE Y UPDATE CON MÁS DE UNA TABLA

Veamos un par de ejemplos.

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | NULL        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |



```
UPDATE LIBROS L
JOIN EDITORIALES E
ON E.Nombre_e = L.Editorial
SET L.Num_paginas = 1000,
    E.Direccion = NULL
WHERE E.Nombre_e = 'Rama';
```

Actualiza con el valor 1000 el número de páginas en la tabla LIBROS para aquellos que son de la editorial Rama y la Dirección en la tabla EDITORIALES.



| Codigo | Titulo          | Num_paginas | Editorial    |
|--------|-----------------|-------------|--------------|
| 1      | Int. Artificial | 50          | Paraninfo    |
| 2      | Virus Informát. | 50          | NULL         |
| 3      | Sist. Informac. | 1000        | Rama         |
| 4      | Turbo C++       | NULL        | Mc Graw-Hill |

| Nombre_e        | Direccion     | Pais   | Ciudad      |
|-----------------|---------------|--------|-------------|
| Mc Graw-Hill    | Basauri, 17   | España | Madrid      |
| Paraninfo       | Virtudes, 7   | España | Madrid      |
| Rama            | Canillas, 144 | España | Madrid      |
| Universal Books | Brown Sq. 23  | EEUU   | Los Ángeles |

# Preguntas

